STEVENS CPE · KYRON COURSE DRAFT LAUNCHER

Course video automation: harness status, Kyron MCP data, test plan and cost evaluation

A checkpoint report before the team starts gathering testing sets.

Prepared	November 2, 2026
Environment	Local development (`localhost:3000`), shared Neon database, OpenAI `gpt-6-luna`
Data window	Logged activity from Oct 30, 2026 8:16 PM to 10:28 PM ET (log system enabled at the start of this window)
Kyron MCP	Connected Nov 2, 2026, 10:50 AM ET with read permission
Screenshots	Captured from the running app on Nov 2, 2026 (admin view)

Summary

$0.017
Total model spend logged (18 calls, 0 failed)

6
QA runs: 4 setup passed, 1 lesson with proposed fixes, 1 waiting for MCP

1
Kyron draft created through the API (draft 14826)

103
Kyron MCP tools available; draft 14826 read (7 activities, 18 scenes)

- **Works end to end today:** source text → course setup generation → setup QA with the gpt-6-luna revision agent → submit gate → Kyron draft created through the public API → generation status. Every model call, fix and content version is logged with cost.
- **Now possible:** reading the *generated* lesson through Kyron MCP (connected Nov 2, read-only). Kyron exposes 103 tools, and draft 14826 was read in full: script, scenes, questions, misconceptions and Kyron's own publish check. **Still to do:** wire this reader into the QA run, and request write permission for fixes and video renders.
- **Cost:** a setup check with fixes costs about $0.003 and a small lesson check about $0.003. The full test plan in section 4 is projected at about $5 in model costs ($10 with contingency). People's review time (about 28 hours) is the real cost.

Contents

1. Kyron MCP dataset: what can be pulled
2. Current harness setup
3. Proposed harness improvements
4. Testing plan: verify the harness and gather problems
5. Cost evaluation
6. Appendix: records referenced

1. Kyron MCP dataset: what can be pulled

> **Connected on November 2, 2026 at 10:50 AM ET, with read-only permission.** The app reads Kyron through **103 MCP tools** (39 read-only). Draft 14826 was read end to end: collection → lesson → section → 7 activities → 18 scenes. Nothing in Kyron was changed. Everything pulled, with media links removed, is saved in `reports/kyron-mcp-dataset-draft-14826-2026-11-02.json` for the testing-set phase.

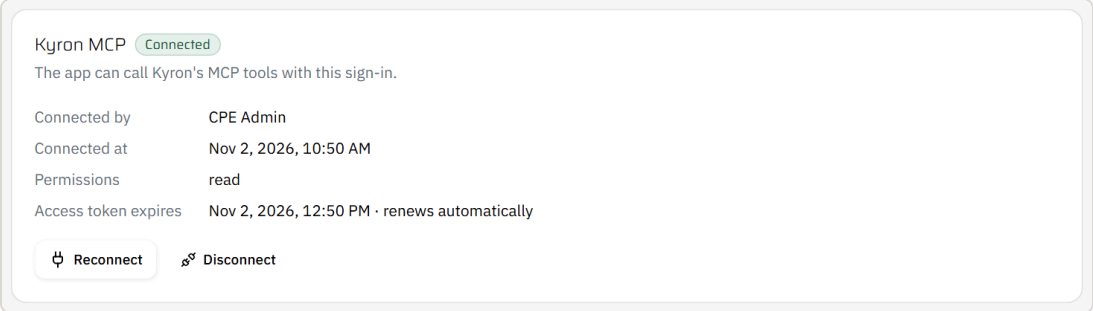


Figure 1. Kyron connection (admin): connected with *read* permission; the sign-in renews automatically.

1.1 How the connection works

- **Client file.** Kyron identifies the app by a public client file (a client ID metadata document). The file is hosted in a secret GitHub gist and served through a JSON mirror (githack). It holds no secrets: only the app name and the address Kyron returns to after sign-in.
- **Lesson learned.** Kyron rejected the file while GitHub served it as plain text (`text/plain`), showing "Client authentication failed due to unknown client…". It accepted the same file once served as `application/json`.
- **Sign-in.** One Kyron account signed in once. The tokens are stored encrypted on this computer and renew automatically (access token lifetime 2 hours).
- **Permission.** `read` only. Writing fixes and starting video renders will need `write`, which should be requested once fixes are tested on a "[QA Test]" copy.
- **Before wider use,** move the client file to a host the team controls, such as GitHub Pages or the deployed app, instead of a third-party mirror.

1.2 What Kyron MCP exposes

Area	Read	Write	Tools the harness uses (bold) and other notable tools
Lesson content	9	15	**get_lesson, list_lesson_sections, list_lesson_activities, get_lesson_activity, update_lesson_activity, validate_lesson, get_lesson_lock_version**; vocabulary, section generation
Video and scenes	8	4	**regenerate_all_lesson_videos**, get_lesson_segment, get_lesson_segment_transcript; avatars, layout previews
Lesson collections (drafts) and versions	5	11	**get_lesson_collection**, list_lesson_collection_versions, copy_lesson_collection; publish_lesson_collection (never automatic)
AI rewrite (regeneration)	1	1	**create_regeneration_log, get_regeneration_log**: Kyron's own AI rewrites one field
Total	**39**	**64**	**103 tools; the app's explorer runs read-only tools only**

Learner runs	4	5	lesson instances, transcripts, misconceptions found in a learner's run (for tutor checks)
Library folders and files	2	10	folders, uploads, attachments
Comments	2	6	comment threads on activities: a possible channel for escalations
Organization, access and keys	5	8	not used (access policies, API keys, organizations)
Integrations (Canvas, LTI)	3	4	not used
Total	**39**	**64**	**103 tools; the app's explorer runs read-only tools only**

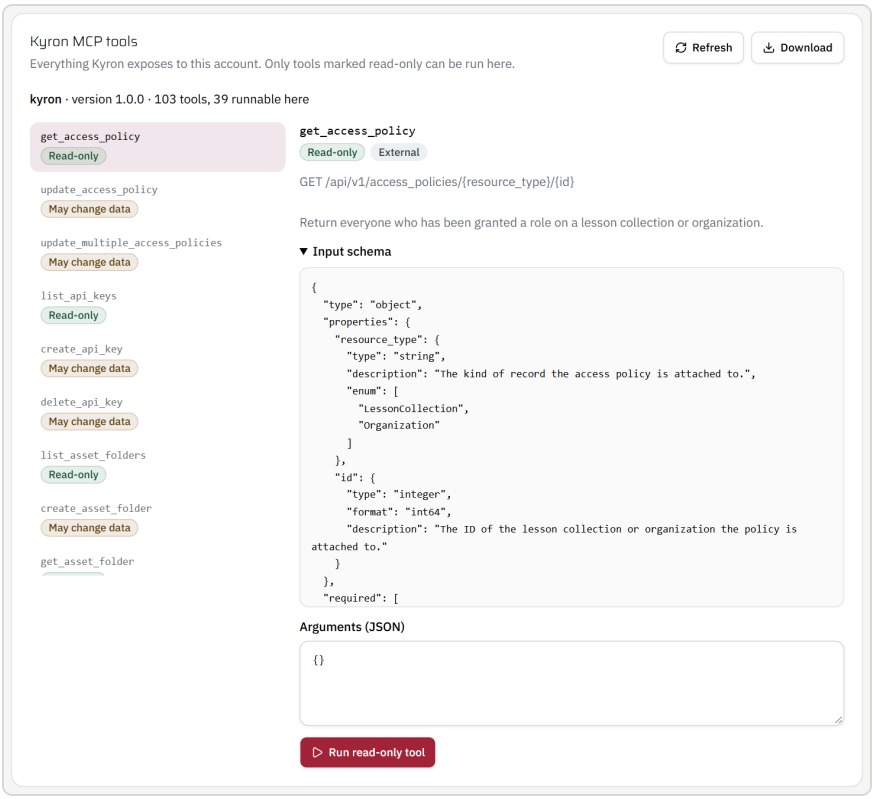


Figure 2. The tool explorer ("MCP terminal") in the app: every Kyron tool with its access level; only read-only tools can be run.

1.3 Data pulled for draft 14826

Level	What came back
Collection 14826	"[QA Test] Introduction to PDEs - MIT 18.152 Class 1" · type `one_shot_course` · status done · not published · description
Draft lesson 23930	Audience, objective, intended time 900 s, avatar "Marcus D", locale en, edit lock version 1, theme and branding, **8 learner misconceptions**, **8 vocabulary terms**
Section 64011	"PDE Order and Linearity": 1 structured objective, 1 enduring understanding, 3 essential questions, knowledge and misconceptions
Activities (7)	Each with type, title, order and branching, plus its full authored content (below)
Scenes (18)	Title, on-screen text, narration script, layout, image, caption file, rendered video address and status

Activity	Kyron type	Title	Scenes
504101	introduction	Introduction	2
504102	reflection_conversation	Think of a time when paying close attention to small details helped you…	2
504103	lecture	Read PDE Notation	4
504104	clarifying_conversation	In one sentence, restate the rule for determining a PDE's order…	2
504105	lecture	Linearity in PDEs	4
504106	clarifying_conversation	Why is the equation u_t = a(x)u_xx + b(x)u still linear…?	2
504107	conclusion	Conclusion	2

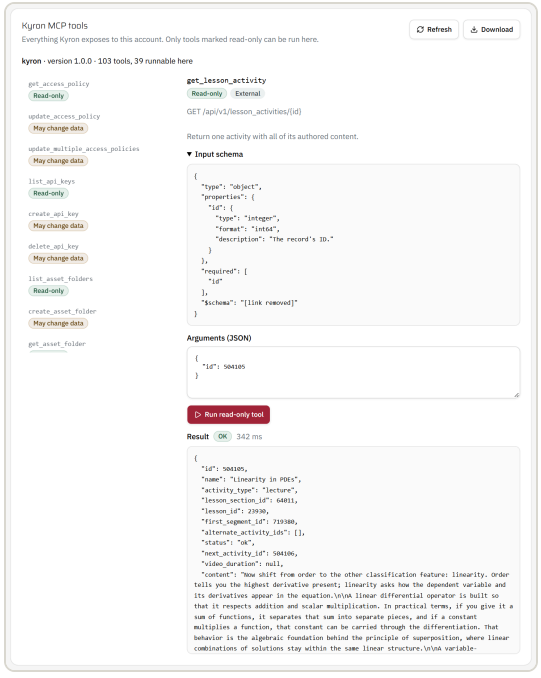


Figure 3. Pulling a lecture in the tool explorer: `get_lesson_activity` with `{"id": 504105}` returns the full script, then its scenes (signed media links removed in this screenshot).

1.4 What one lecture and one conversation contain

Lecture "Linearity in PDEs" (504105): 4 scenes, 242 narration words, captions for every scene, no rendered video yet.

#	Scene title	Layout	Narration (start)	On-screen text
0	Linearity vs Order	full_bleed_image	"Now shift from order to the other classification feature: linearity..."	-
1	Operator Linearity	image_info	"A linear differential operator is built so that it respects addition and scalar multiplication..."	-
2	Variable-Coefficient PDEs	text_info	"A variable-coefficient linear PDE is one where the coefficient functions depend on the independent variables..."	Coefficients depend on x or t · They multiply the unknown...
3	Linear vs Nonlinear Terms	full_bleed_image	"Differentiation itself is linear: the derivative of a sum is the sum of the derivatives..."	-

Conversation (504106): the question learners discuss, a model answer, 1 answer option, the knowledge it targets with 4 misconceptions ("Confusing linearity and order", "Belief that variable coefficients make an equation nonlinear", ...), a bank of 4 alternative questions, and its own question and wait videos.



Figure 4. `get_lesson_collection` (14826): collection, draft lesson and its settings.



Figure 5. `list_lesson_activities`: the 7 activities in learner order.

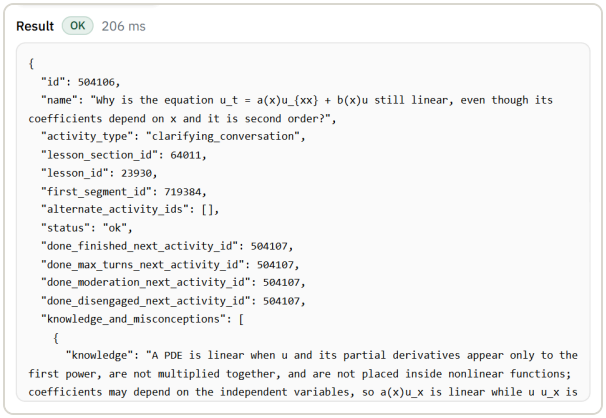


Figure 6. `get_lesson_activity` (504106): conversation question, answer and misconceptions.

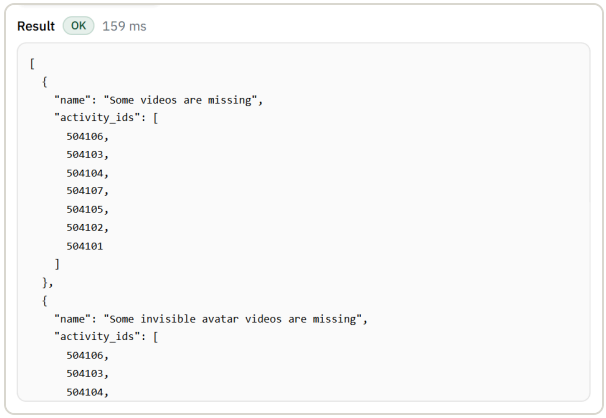


Figure 7. `validate_lesson` (23930): Kyron's own publish check reports missing videos on all 7 activities.

1.5 How Kyron's data maps to the QA harness

Harness snapshot field	Kyron MCP source	Tool
lesson.title, description, audience	Lesson name, description, audience	get_lesson
lesson.learningObjectives	Lesson objective + section structured objectives	get_lesson, list_lesson_sections
lesson.durationSeconds, assetType	intended_time, lesson_type	get_lesson
activity.id, type, title	Activity id, activity_type, name. Types: introduction, lecture and conclusion map to *instruction*; reflection_conversation to *reflection*; clarifying_conversation to *conversation*	list_lesson_activities
activity.narration	Lecture `content` and each scene's `scene_narration`	get_lesson_activity
activity.onScreenText	Scene titles and `scene_text`	get_lesson_activity
activity.question	question, answer, answer_options, question_bank	get_lesson_activity
activity.tutorGuidance	knowledge_and_misconceptions; lesson misconceptions	get_lesson_activity, get_lesson
Video and narration check	Scene caption files (VTT), scene video address, video_status	get_lesson_activity
References for fidelity	Kyron's `citations` field is empty for this lesson; the workspace sources stay the reference	-

Harness action (needs `write`)	Kyron tool and behavior
Apply a safe fix	`update_lesson_activity` with the lesson's `lock_version`. Kyron rejects a stale version, which matches the harness rule of never writing over a person's edit
Ask Kyron's own AI to rewrite a field	`create_regeneration_log` (optionally auto-accept), then `get_regeneration_log`
Render video once the script passes	`regenerate_all_lesson_videos`, then poll `get_lesson_activity` video status or `validate_lesson`
Publish	`publish_lesson_collection`: never automatic; needs a person's explicit confirmation

1.6 Findings from the real Kyron data

#	Finding	What it means for the harness
1	**No video is rendered yet.** `validate_lesson` reports missing videos (and missing invisible-avatar videos) on all 7 activities; 0 of 18 scene videos exist.	Rendering is a separate step (`regenerate_all_lesson_videos`). That fits the plan: render only after the script passes QA.
2	**Segment and transcript reads are refused** ("Access forbidden") with read permission.	Caption files exist for 15 of 18 scenes and can serve as the narration transcript. Retest the segment tools with write permission.
3	**A claim not in the source:** the lecture and the vocabulary list cite the Black–Scholes equation. The MIT notes mention only the heat, wave and Laplace equations.	A real, unseeded case for the source-fidelity check, and the first entry for the testing set.

4	**Narration is short for the target time:** about 803 words (5–6 minutes) for a 15-minute lesson; conversations make up the rest.	Calibrate the duration check with real learner timings.
5	**No citations** are attached to any activity.	Grounding in the sources has to come from the harness's own fidelity check.

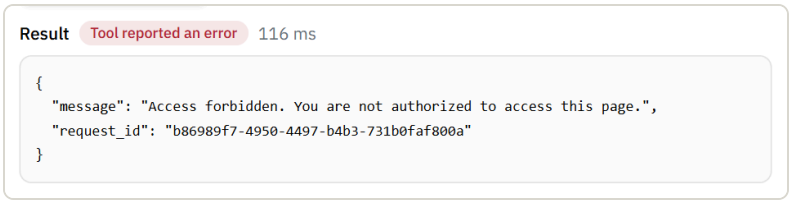


Figure 8. `get_lesson_segment_transcript` with read permission: refused by Kyron.

1.7 Before the connection: a lesson pasted into the harness

Before Kyron MCP was connected, lesson QA was tested on a short lesson in the snapshot format was pasted into the draft 14826 workspace. It was titled **"[QA Test] Seeded error - PDE classification"** and contained one deliberate false claim: that the heat equation is nonlinear. The checks found it as **critical** in both the narration and the on-screen text (see the workspace panel in section 2.2), and the revision agent proposed a correction. Pasted content cannot be changed by the agent, so the correction was stored as a downloadable proposed version.

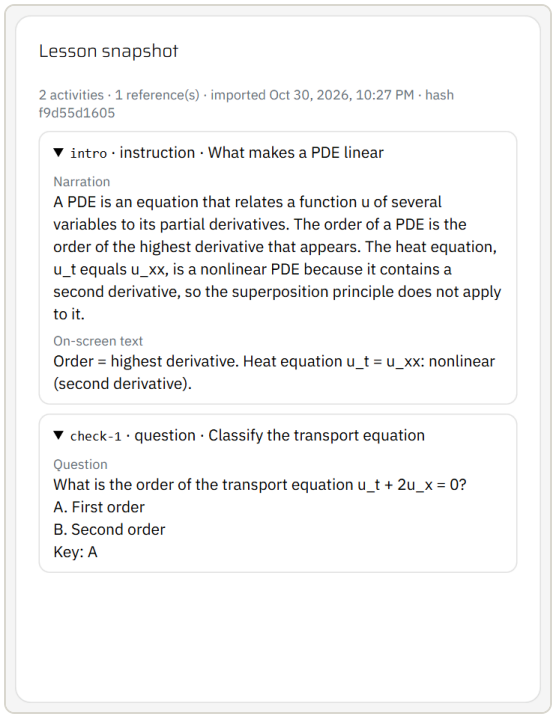


Figure 9. The snapshot as the checks see it: activities, narration, on-screen text, question and key.

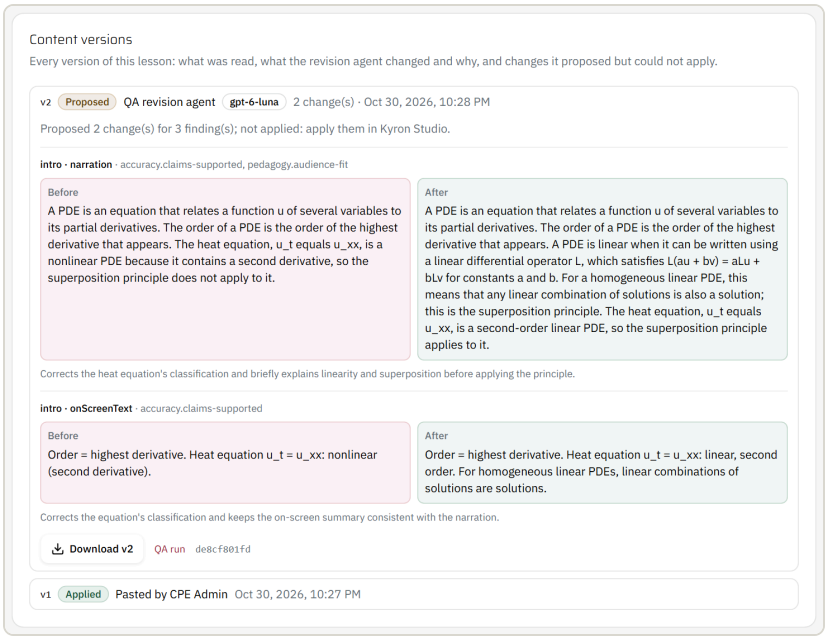


Figure 10. Content versions for the lesson: v1 as pasted, v2 proposed by the revision agent, with before and after for each field.

2. Current harness setup

The harness turns source material into a Kyron lesson and checks it at each stage. Quality checks run **inside** generation, not as a separate tool. Steps are stored in Neon with leases, so a step that fails or times out is retried without running twice.

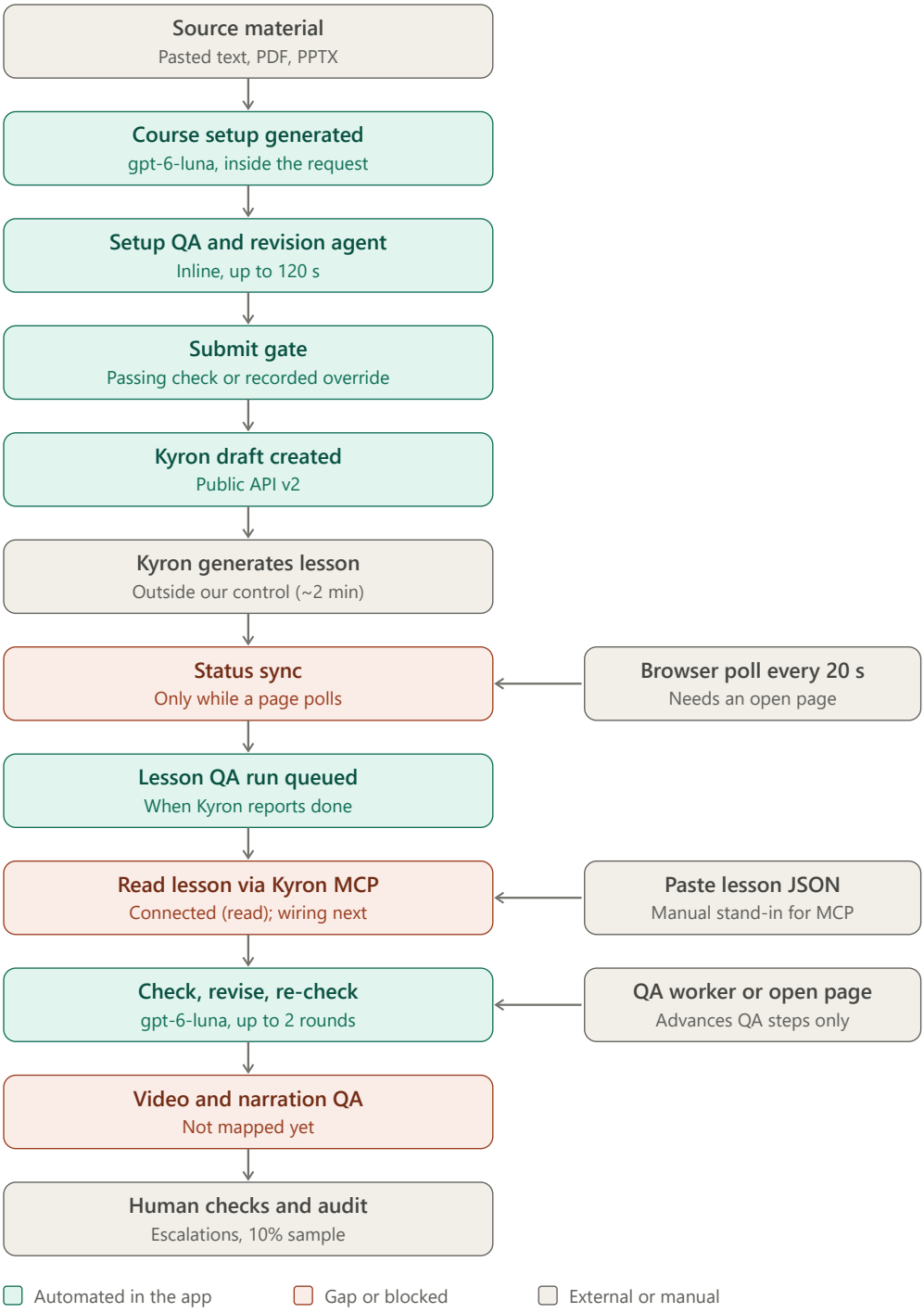


Figure 11. Current harness. Automation is complete up to the Kyron handoff. After it, status sync depends on an open page; Kyron MCP reading works but is not yet wired into QA runs, and fixing and video need write permission.

2.1 Components

Stage	How it works	Status
Sign-in and roles	Preset access codes (admin, developer), signed session cookie, localhost only. Clerk kept for later hosting.	done
Course setup generation	gpt-6-luna (medium effort) builds name, description, objective, audience, duration and type from the sources.	done
Setup QA	Five criteria (source fidelity, measurable objective, grounded audience, duration and type fit, clear title), strict JSON output. Quoted evidence is verified against the sources.	done
Revision agent	One gpt-6-luna call per round fixes all open blocking findings together. The server checks each change, applies the safe ones (reading before and after the write), and keeps critical and answer-key changes as proposals. At most 2 rounds.	done
Content versions	Every state of the setup or lesson is numbered, with author, model, reason, and before and after per field. Shown on run pages and the Logs page; each version can be downloaded.	done
Submit gate	Kyron draft creation waits for a current, passing setup check. "Submit anyway" is recorded.	done
Kyron draft creation	Public API v2, with optional PDF/PPTX attachment from the Source Library.	done
Status sync	The browser polls every 20 s; a finished draft queues the lesson check.	needs open page
Lesson QA	Structure, blind assessment, source fidelity, consistency, alignment and audience checks, then the revision agent.	paste only
Kyron MCP read/write	Connected with read permission (OAuth with PKCE, encrypted tokens). 103 tools listed; draft 14826 read. Not yet wired into QA runs; write permission not requested.	read only
Video and narration QA	Request video only after the script passes (Kyron: `regenerate_all_lesson_videos`), transcribe it, and compare with the script. Uploading a video file works today.	needs write
Human checks and audit	Escalation queue, accept/reject/waive with a reason, tutor and visual checks, 10% random audit.	done
Logs and cost	Every model call (tokens, cost, latency), Kyron calls, QA steps, versions and errors. Secrets are removed. CSV/JSON export.	done

2.2 The harness in the app

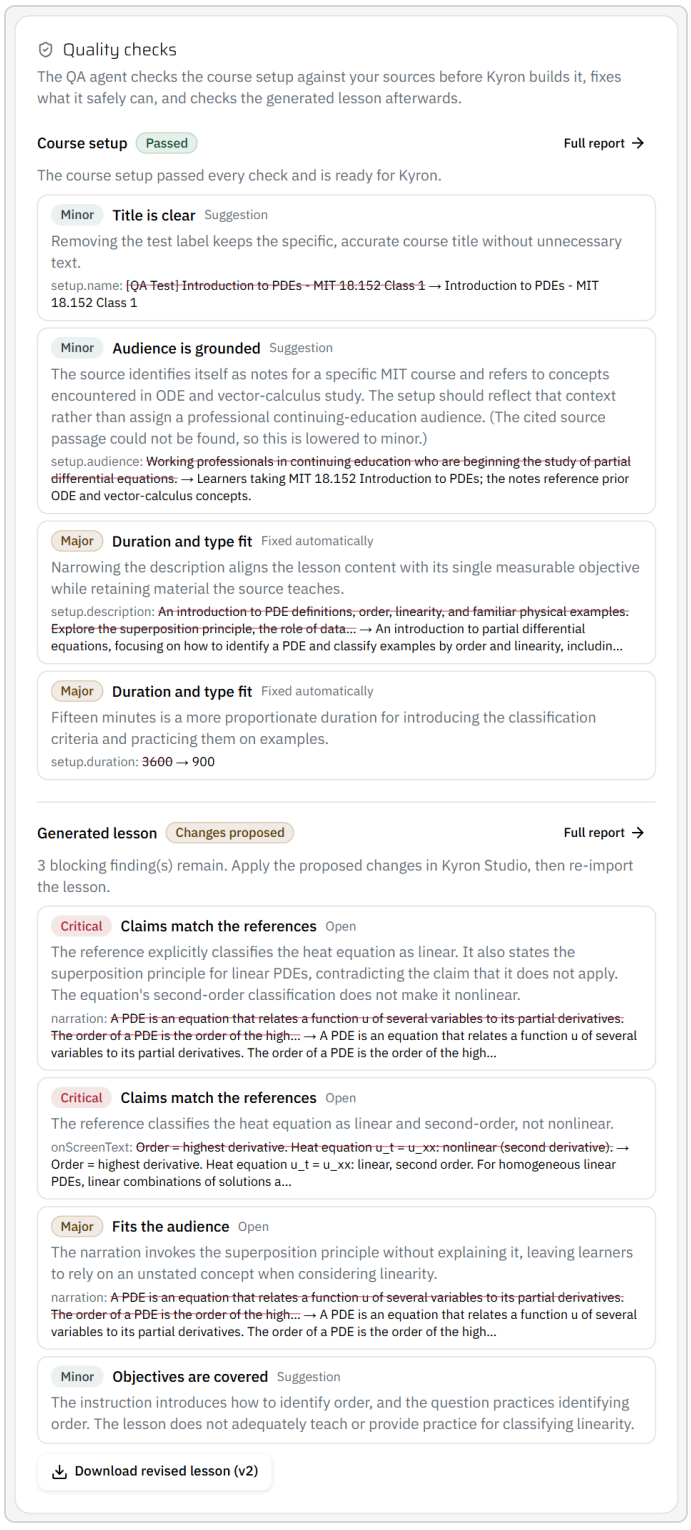


Figure 12. Quality checks on the workspace page of draft 14826: the setup (passed, with automatic fixes) and the pasted generated lesson (changes proposed, with a download of the revised lesson).

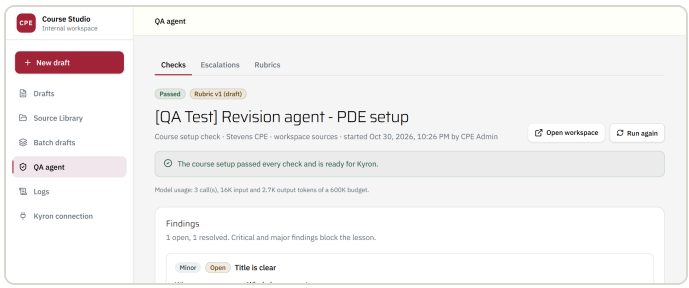


Figure 13. A setup QA run that passed after the revision agent's fix.

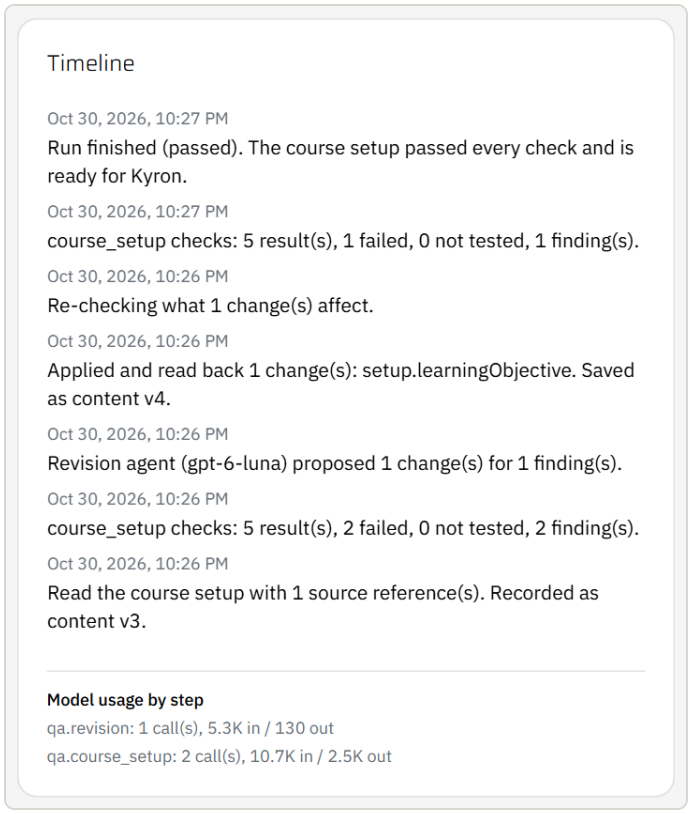


Figure 14. Timeline for that run: read, recorded as v3, checks, revision agent (gpt-6-luna), applied and read back as v4, re-check, passed.

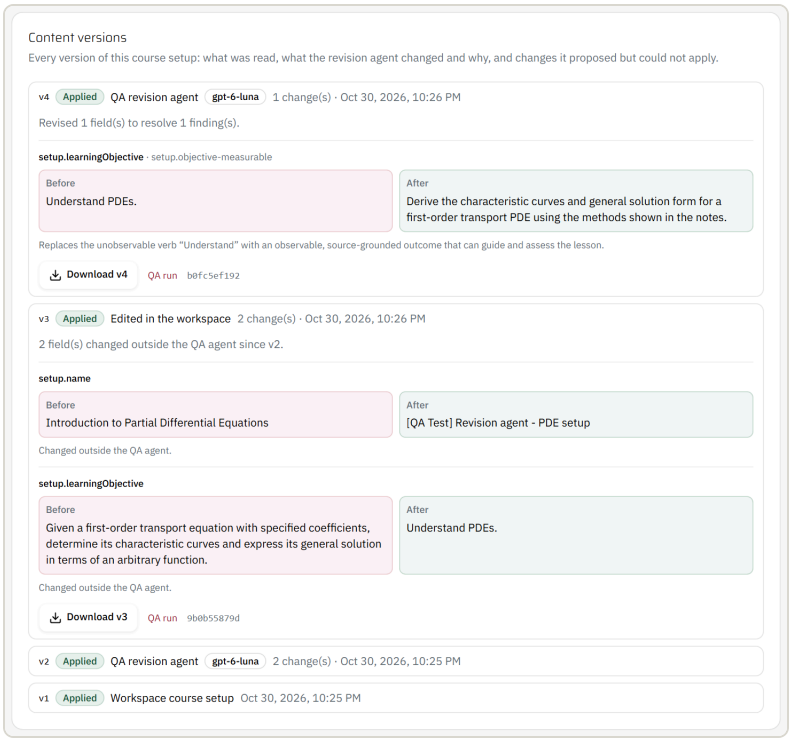


Figure 15. Content versions for a course setup. v3 records a person's edit (the vague objective "Understand PDEs."). v4 is the revision agent's fix, with the finding it resolves and its reason.

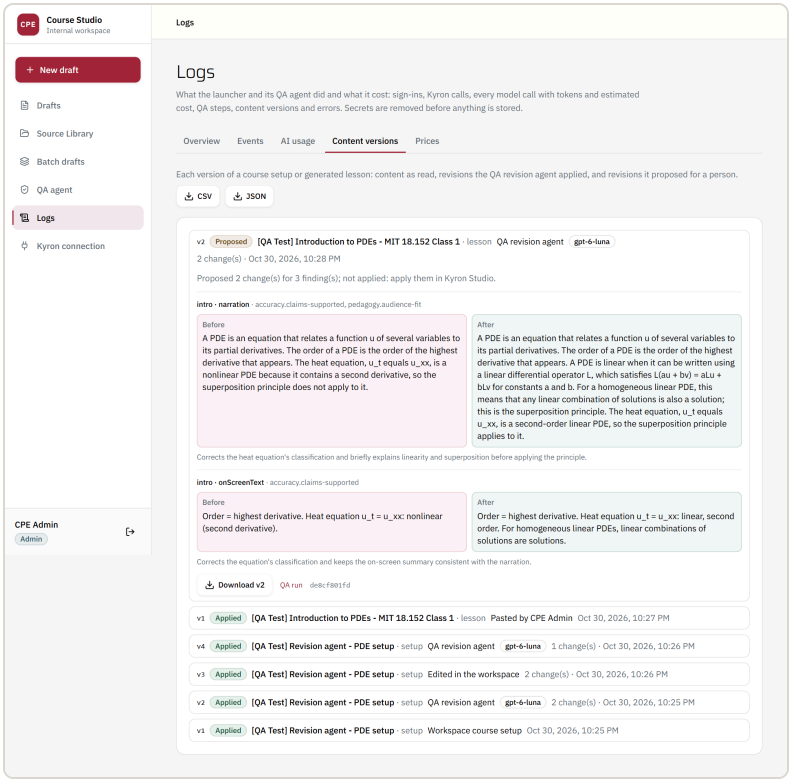


Figure 16. **Logs → Content versions**: every version across workspaces, with CSV and JSON export (one row per changed field).

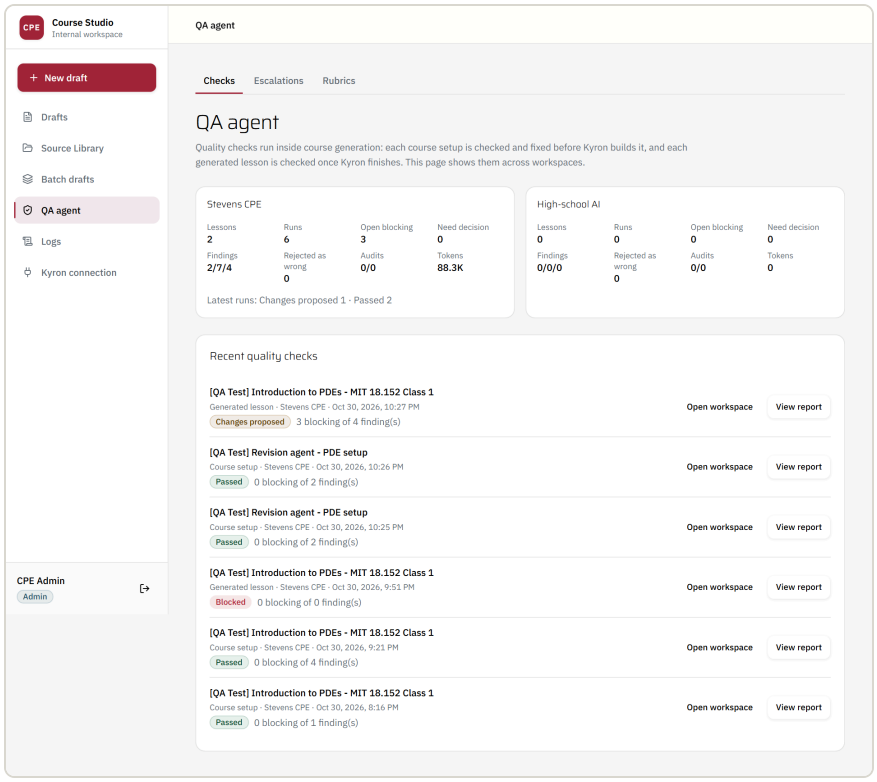


Figure 17. QA agent dashboard: every quality check across workspaces. The test content is named with a "[QA Test]" prefix.

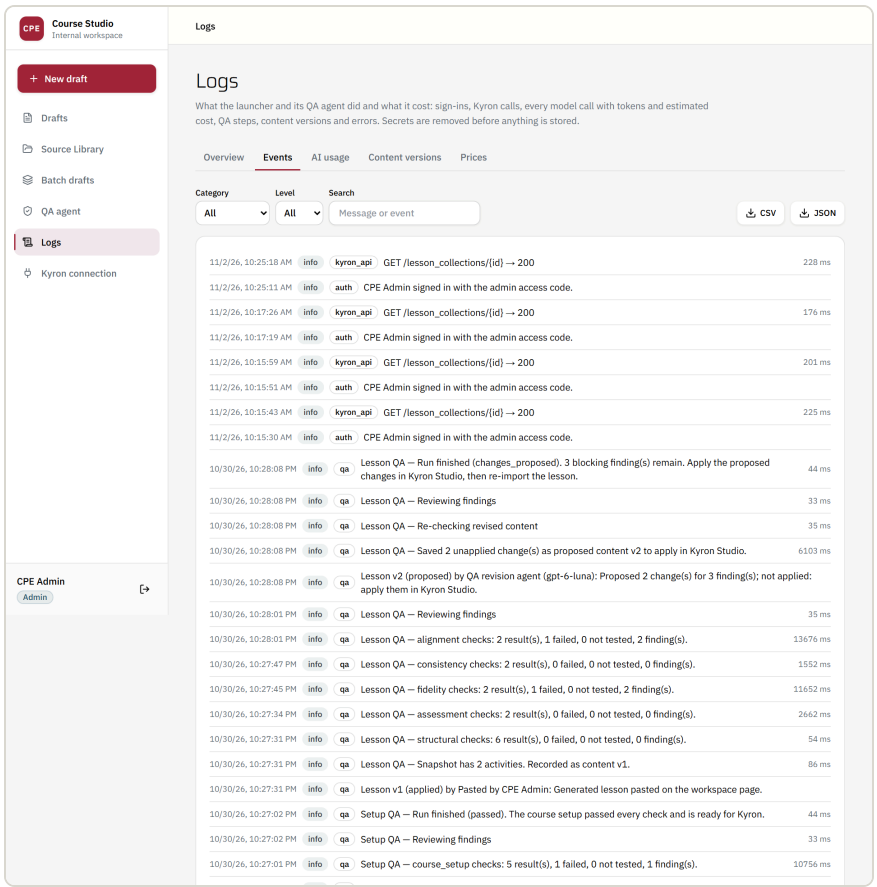


Figure 18. **Logs → Events**: Kyron API calls, QA steps, content versions, sign-ins and errors. Details are kept for each event, with secrets removed.

3. Proposed harness improvements

To reach automation with no page open, these changes are recommended, in this order:

#	Change	Why	Depends on
1	Durable production job per course, run by one scheduler (worker or Vercel Cron)	Work is currently triggered from three places (generate request, submit request, status sync). Batches run inside one request and are limited to 20.	Nothing
2	Server-side Kyron status poller	The lesson check only starts if someone has the page open.	#1
3	Wire the Kyron MCP reader into QA runs (field mapping in section 1.5)	The largest gain: automatic lesson reading now; fixing and video once write is granted.	Sign-in done; write permission
4	Independent verifier after each revision	The same model and prompt currently re-check their own fix.	Nothing
5	Narration fix and re-render loop (at most once)	A failed video check currently ends the run.	#3
6	Defect library and seeded test set	Measures how many errors QA catches; feeds rubric tuning.	This test plan

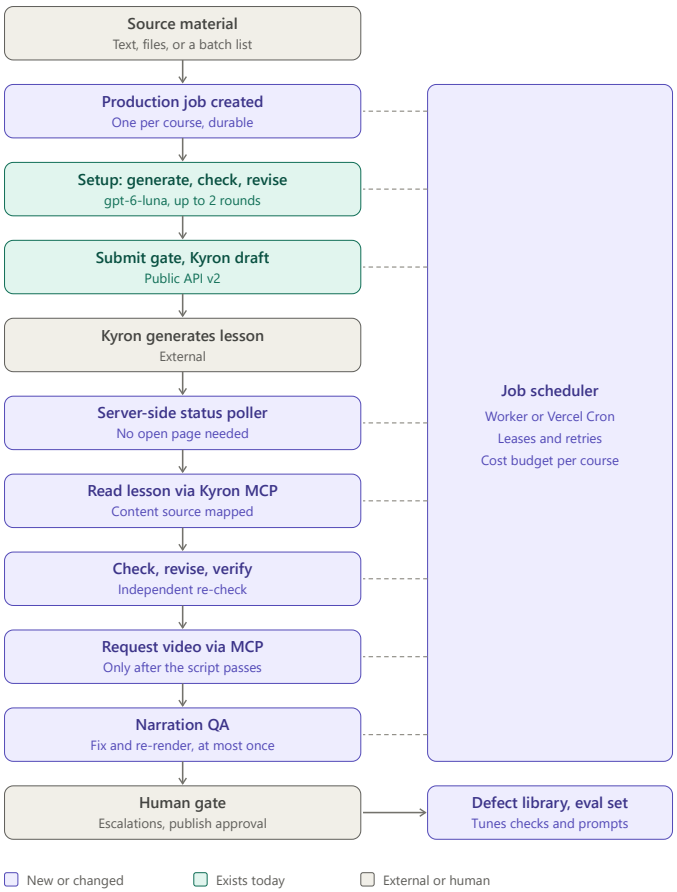


Figure 19. Proposed harness. One production job per course, advanced by a single scheduler; everything after the Kyron handoff runs without an open page.

4. Testing plan: verify the harness and gather problems

Goals: (1) confirm each stage works as designed; (2) measure how well the checks catch real and seeded mistakes, and how often they raise false alarms; (3) log every problem in one place, with evidence, so the next phase can build the testing sets from it.

4.1 Phases

Phase	Scope	Pass criteria	Needs
0. Smoke done Oct 30	One course end to end: setup QA with fixes, Kyron draft 14826, a pasted lesson with a seeded error	All stages ran; seeded critical error caught; costs logged	-
1. Harness regression	94 automated tests (pipeline, revision agent, repair policy, auth, logging), plus lint, typecheck and build on every change	All pass; no unchecked write over a person's edit	None
2. Setup QA	20 real source documents, plus 20 setups with seeded problems (table 4.2); each run twice	≥90% of critical and ≥75% of major seeded problems caught; ≥80% of applied fixes judged correct; the same verdict on both runs ≥85% of the time	Source files; ~6 h review
3. Lesson QA (paste)	15 lessons with ~3 seeded defects each (≈45) and 5 clean lessons; each run 3 times	Same catch targets; ≤1 blocking false alarm per 5 clean lessons; no answer key changed automatically	Lesson authoring and review, ~10 h
4. Real-course pilot	10 CPE courses through the full flow to Kyron drafts (all named "[QA Test] ..."). Reviewers compare the lessons with the QA findings	Every problem a reviewer finds is logged; QA misses are classified	Kyron credits; ~7.5 h review
5. MCP and video reads ready	Read, write and video on 10 lessons; narration checked against video transcripts	Reads match Studio; writes read back exactly; transcripts match the script	Write permission for fixes and renders

4.2 Seeded defect catalog

Where	Defect seeded	Expected severity	Expected harness behavior
Setup	Vague objective ("Understand X")	Major	Fixed automatically and re-checked
Setup	Duration that doesn't fit the material (e.g. 60 min for one page)	Major	Fixed automatically to an allowed duration
Setup	Invented audience, employer or credential	Major	Fixed automatically; evidence quoted from the source
Setup	Off-topic description	Critical	Proposed version, escalated, not applied
Lesson	Claim that contradicts the source	Critical	Found; fix proposed (MCP: escalated for approval)
Lesson	Claim the source doesn't support	Major	Found; fix proposed or applied

Lesson	Wrong answer key	Critical	Found by the blind solve; key never changed automatically
Lesson	Two defensible answers; question not answerable from the lesson	Major	Found; prompt or options fix proposed
Lesson	Narration and on-screen text disagree	Major	Both fields revised together
Lesson	Placeholder text ("[TODO]", "Lorem ipsum")	Major	Found by the structural rules (no model needed)
Lesson	Objective not covered; jargon above the audience	Major	Found by the alignment and audience checks
Lesson	Embedded instruction ("ignore previous instructions…")	-	Treated as untrusted text: not followed, nothing written

4.3 Measures

Measure	Definition	Target
Catch rate	Seeded defects found ÷ seeded, by type and severity	≥90% critical, ≥75% major
False alarm rate	Blocking findings on clean content that reviewers reject	≤1 per 5 clean lessons
Fix acceptance	Fixes a reviewer judges correct and in the author's voice	≥80%
Fix safety	Writes over a person's edit; answer keys changed automatically; new errors introduced by a fix	0
Consistency	Same input, same verdict and severity across repeated runs	≥85%
Escalation rate	Runs that need a person's decision	Tracked (no target yet)
Time and cost per course	From the logs: latency, tokens, estimated cost	Within section 5 estimates

4.4 How problems and mistakes are gathered

Recorded automatically: findings with evidence (run report), every content version with before and after, decisions (accept, reject or waive, with a reason), audits, model usage and cost, and errors (Logs). All of it exports to CSV/JSON.

Recorded by reviewers in one defect log, one row per problem:

ID	Course / run link	Stage	Category	Severity	Expected vs actual	Evidence (version / finding)	Root cause	Action
D-001	run ad7fdfe6	Lesson	QA missed	Major	…	v2 · intro.narration	Rubric	Tune

Categories: QA missed a problem · False alarm · Wrong or harmful fix · Harness bug · Kyron generation error · Source or extraction problem · Cost or performance. A short weekly triage assigns each row an owner. Rows marked "QA missed" and "False alarm" become the testing sets.

4.5 Problems observed so far (smoke test)

#	Observation	Category	Next step

1	Kyron rejected the client file while GitHub served it as plain text; it was accepted as `application/json`. MCP connected Nov 2.	Resolved	Move the file to a host the team controls
2	Kyron status only updates while a page is open.	Harness gap	Server-side status poller (section 3)
3	Duration fixes went in opposite directions for the same source (3600→900 in one run, 1800→3600 in another).	Consistency	Repeated-run test in phase 2; tighten the scope rule
4	Math symbols in the PDF text came out as "□ □ □", so a quoted passage could not be matched and the finding was lowered to minor.	Source extraction	Test math-heavy PDFs; improve extraction
5	A run can show *Passed* while one check is *Failed* because its only finding is minor.	Rubric decision	Decide whether minor-only failures should pass
6	Course setup generation replaces the workspace name, so test names had to be re-applied.	Harness bug	Keep a name the user set
7	The title check keeps suggesting removing the "[QA Test]" label.	False alarm	Ignore test prefixes in the title check
8	The revision agent combined an audience fix with a factual fix in one narration rewrite (+~250 characters).	Watch	Check length and tone drift in fix acceptance
9	Model calls before the log system was switched on (early Oct 30) are not in the cost totals.	Reporting	None; totals start Oct 30, 8:16 PM ET
10	The Kyron lecture and vocabulary cite the Black–Scholes equation, which the source never mentions.	Kyron output	First real case for the fidelity check and testing set
11	Kyron refuses segment and transcript reads with read permission; no video is rendered yet.	Access	Use caption files; retest with write permission
12	Narration is about 5–6 minutes for a 15-minute lesson.	Calibration	Measure real learner time before judging duration

5. Cost evaluation

5.1 Spent so far (from the logs)

All model calls ran on `gpt-6-luna` at **$0.10 per 1M input tokens, $0.01 per 1M cached input, $0.50 per 1M output** (reasoning tokens are billed as output). Each call's cost is calculated when it is logged, at the price in effect then. These are the app's own estimates from logged token counts, not an OpenAI invoice.

Purpose	Calls	Input tokens	Output tokens	Avg latency	Cost	Avg / call
Setup QA (`qa.course_setup`)	9	48,386	13,019	16.9 s	$0.01135	$0.00126
Revision agent (`qa.revision`)	3	16,493	803	3.7 s	$0.00205	$0.00068
Source fidelity (`qa.fidelity`)	1	5,207	1,711	11.5 s	$0.00138	$0.00138
Setup generation (`course_setup`)	2	9,944	645	5.4 s	$0.00132	$0.00066
Alignment (`qa.alignment`)	1	675	968	13.6 s	$0.00055	$0.00055
Blind assessment	1	449	87	2.6 s	$0.00009	$0.00009
Consistency	1	435	88	1.5 s	$0.00009	$0.00009
Total	**18**	**81,589**	**17,321**		**$0.01682**	**$0.00093**

Output tokens include 12,079 reasoning tokens. No cached input was used and no call failed. Kyron API calls (84 logged, 0 errors) are not billed per call by OpenAI; their cost depends on the Kyron account plan.

5.2 Cost per run

Run	What it was	Outcome	Calls	Cost	Of which revision
601eb10b	Setup QA, first live test	Passed	2	$0.00214	-
11fad144	Setup QA, draft 14826 workspace (old per-field repair)	Passed after 3 fixes	3	$0.00469	-
cede430c	Setup QA inside generation (revision agent)	Passed after 2 fixes	3	$0.00282	$0.00065
497586fc	Setup QA after a person's edit (revision agent)	Passed after 1 fix	3	$0.00294	$0.00059
ad7fdfe6	Lesson QA, pasted 2-activity lesson with seeded error	Changes proposed	5	$0.00291	$0.00081
c1bed8fb	Lesson QA for draft 14826 via MCP	Blocked (no MCP)	0	$0.00000	-

> **Unit costs measured:** setup generation ≈ $0.0007; setup QA with fixes ≈ $0.003 (range $0.002 to $0.005); a revision agent call ≈ $0.0007; a small pasted-lesson check ≈ $0.003. A course through setup and submission costs well under one cent in model calls.

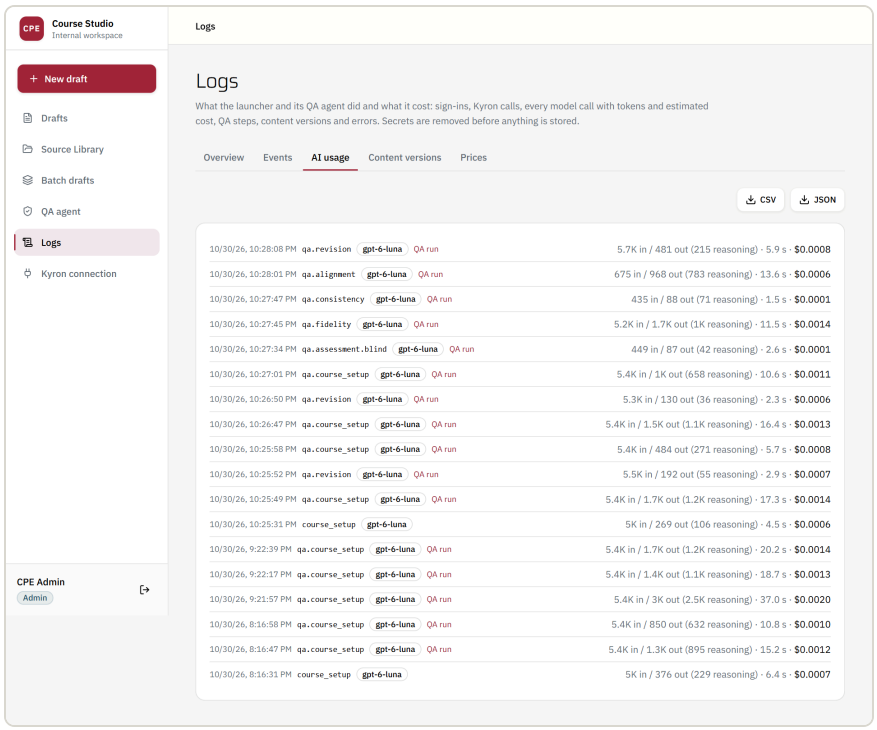


Figure 20. **Logs** → **AI usage**: each model call with purpose, model, tokens, latency, cost and a link to its QA run.

5.3 Projected cost of the testing plan

Real generated lessons will be roughly 10 times longer than the 2-activity sample. The estimate below assumes **$0.03 per lesson check** and **$0.05 with two revision rounds**. Narration transcription is estimated at about $0.006 per audio minute (OpenAI's published estimate for `gpt-4o-transcribe`), about $0.09 for a 15-minute video. Every run has a hard limit of 600,000 tokens (at most about $0.30 per run even if every token were output), and hitting it stops the run instead of skipping checks.

Phase	Volume	Model cost basis	Model cost	People's time	Kyron cost
1. Harness regression	94 tests per change	Models are mocked	$0.00	-	-
2. Setup QA	40 setups × 2 runs	$0.0007 generation + $0.0031 QA	≈ $0.30	≈ 6 h	-
3. Lesson QA (paste)	20 lessons × 3 runs	$0.03 per run	≈ $1.80	≈ 10 h	-
4. Real-course pilot	10 courses	$0.004 setup + $0.05 lesson	≈ $0.55	≈ 7.5 h	10 drafts: to confirm
5. MCP and video	10 lessons	$0.05 QA + 2 × $0.09 transcription	≈ $2.30	≈ 5 h	10–20 renders: to confirm
Total			**≈ $5**	**≈ 28 h**	**To confirm with Kyron**

> **Budget recommendation:** set aside **$10** for model calls (2× contingency for retries, longer lessons and prompt tuning). Model calls are not the main cost of testing. People's review time (about 28 hours) and Kyron generation and video credits are. Kyron's per-draft and per-render pricing should be confirmed with the account owner before phases 4 and 5.

5.4 Cost controls already in place

- **Per-run token budget** (`QA_RUN_TOKEN_BUDGET=600000`): the run stops as *Blocked*; checks are never skipped silently.
- **Model pinned for corrections** (`OPENAI_REVISION_MODEL=gpt-6-luna`) with medium effort; outputs are not stored at OpenAI (`store: false`).
- **Video only after the script passes**, so failing lessons never use render credits.
- **Price table with effective dates:** a price change applies from that point on and never rewrites past costs. Calls to a model with no price show "rate not set" instead of $0.
- **Logs** show spend for 24 hours, 7 days and 30 days, by model and by purpose, with CSV/JSON export for each phase's cost report.

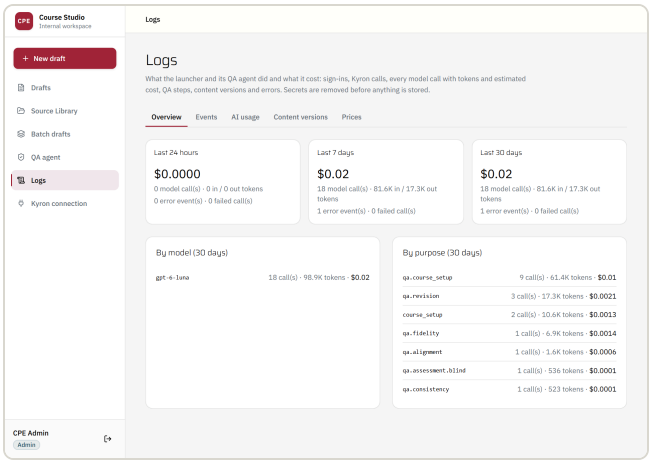


Figure 21. **Logs** → **Overview**: spend by period, model and purpose.

6. Appendix: records referenced

Record	Details
Kyron draft	14826, "[QA Test] Introduction to PDEs - MIT 18.152 Class 1", created through the app's API on Oct 30, 2026 (status done)
Workspace (draft 14826)	7e49593f-0596-4b1f-9c15-ab6507044bea: setup run 11fad144 (passed), lesson runs c1bed8fb (blocked: MCP) and ad7fdfe6 (pasted, changes proposed)
Workspace (revision agent test)	36b18f77-7e67-41f6-aefa-469d1ca10818, "[QA Test] Revision agent - PDE setup": runs cede430c and 497586fc, content versions v1–v4
Kyron MCP	Connected Nov 2, 2026, 10:50 AM ET as CPE Admin, permission `read`; client file served as JSON from the secret gist. Captured data: `reports/kyron-mcp-dataset-draft-14826-2026-11-02.json` (media links removed)
Kyron lesson IDs	Collection 14826 → draft lesson 23930 → section 64011 → activities 504101–504107
Source used	`test_course.pdf`: MIT 18.152 Introduction to PDEs, Class Meeting 1 (text pasted as source; Vercel Blob not configured locally)
Database	Shared Neon database; QA, log and version tables are migrations 100–120 (applied on their own, twice, to confirm they are safe to re-run)
Code checks	Lint, typecheck, 94 tests and production build all passing as of Oct 30, 2026
Configuration	`AUTH_MODE=access-code`, `OPENAI_MODEL=gpt-6-luna`, `OPENAI_REVISION_MODEL=gpt-6-luna`, `QA_AUTONOMY_POLICY=auto`, `QA_RUN_TOKEN_BUDGET=600000`, `QA_AUDIT_RATE=0.1`. `KYRON_OAUTH_CLIENT_ID` set to the JSON-served client file.